

CS2601 Linear and Convex Optimization

Homework 6 Solution

Due: 2021.11.11

For this assignment, you should submit a **single** pdf file as well as your source code (.py or .ipynb files). **The pdf file should include all necessary figures, the outputs of your Python code, and your answers to the questions.** Do NOT submit your figures in separate files. Your answers in any of the .py or .ipynb files will NOT be graded.

In this assignment, you will implement gradient descent with constant step size. First complete the function `gd_const_ss` in `gd.py`.

1. Consider the following quadratic program,

$$\min_{\mathbf{x} \in \mathbb{R}^2} f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Q} \mathbf{x} \quad (1)$$

where

$$\mathbf{Q} = \begin{pmatrix} \gamma & 0 \\ 0 & 1 \end{pmatrix}.$$

with $\gamma > 0$. In terms of coordinates,

$$f(x_1, x_2) = \frac{\gamma}{2} x_1^2 + \frac{1}{2} x_2^2.$$

We know the optimal solution is $\mathbf{x}^* = \mathbf{0}$ and $f^* = f(\mathbf{x}^*) = 0$.

- (a). What is the smallest L such that f is L -smooth?
- (b). What is the largest m such that f is m -strongly convex?
- (c). Suppose $\gamma = 0.1$. Complete `p1.py` and run the gradient descent algorithm you implemented with $\mathbf{x}_0 = (1, 1)$ and step sizes 2.2, 1, 0.1, 0.01. (Note: for step size 2.2, you may want to limit the maximum number of iterations to a small number, say 10.) Does it converge in each case? When it does converge, how many iterations does it take? In each case, plot the 2D trajectory of the sequence $\mathbf{x}_k$ and the function values $f(\mathbf{x}_k)$. You can use the function provided in `utils.py` or write your own code. Note that $f(\mathbf{x}_k)$ is also the suboptimality gap since $f(\mathbf{x}^*) = 0$.
- (d). For $\gamma = 1, 0.1, 0.01, 0.001$, run gradient descent with step size 1. How many iterations do you need in each case? How does the number of iterations change as γ decreases? You don't need to show the plots.

Solution. A possible implementation is given below.

```

def gd_const_ss(fp, x0, stepsize, tol=1e-5, maxiter=100000):
    x_traces = [np.array(x0)]
    x = np.array(x0)

    for k in range(maxiter):
        grad = fp(x)
        if np.linalg.norm(grad) < tol:
            break

        x -= stepsize * grad
        x_traces.append(np.array(x))

    return x_traces

```

- (a). The eigenvalues of $\mathbf{Q}$ are γ and 1, so $L = \max\{\gamma, 1\}$.
- (b). $m = \min\{\gamma, 1\}$.
- (c). The algorithm diverges for step size 2.2 and converges in all other cases. The number of iterations is shown in Table 1. The plots are shown in Figures 1 - 4. When step size is 0.22, $f(\mathbf{x}_k)$ grows exponentially with k . In all other cases, $f(\mathbf{x}_k)$ decays exponentially.

step size	number of iterations
0.1	88
0.01	917
0.001	9206

Table 1: $\gamma = 10$

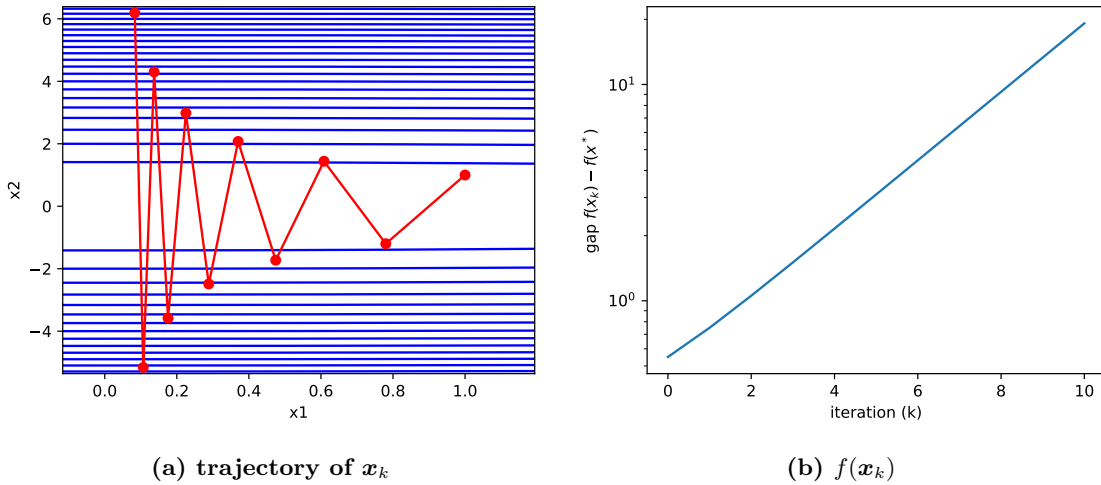
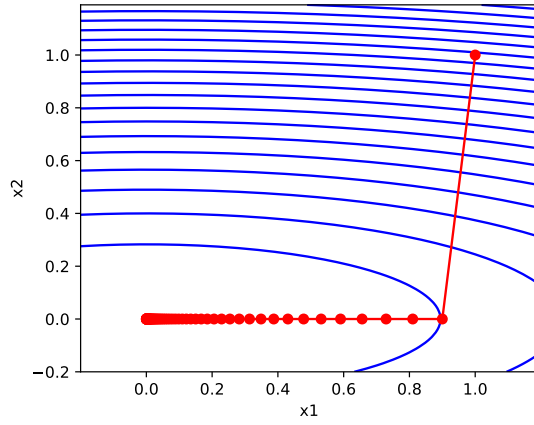
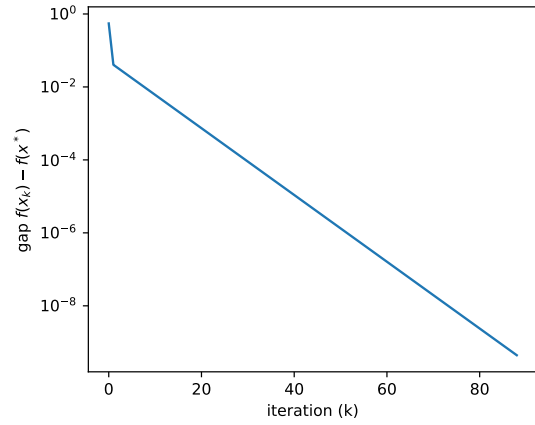


Figure 1: $\gamma = 0.1$, step size 2.2

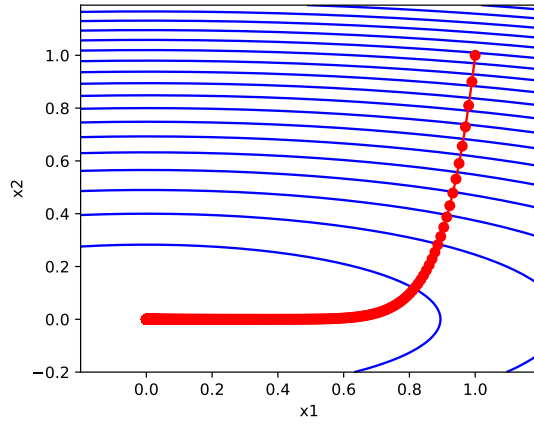


(a) trajectory of x_k

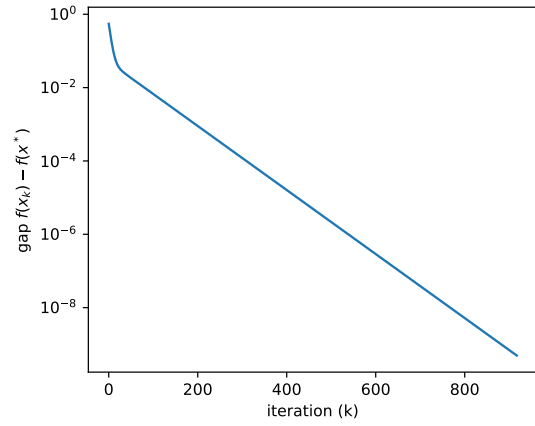


(b) $f(x_k)$

Figure 2: $\gamma = 0.1$, step size 1



(a) trajectory of x_k



(b) $f(x_k)$

Figure 3: $\gamma = 0.1$, step size 0.1

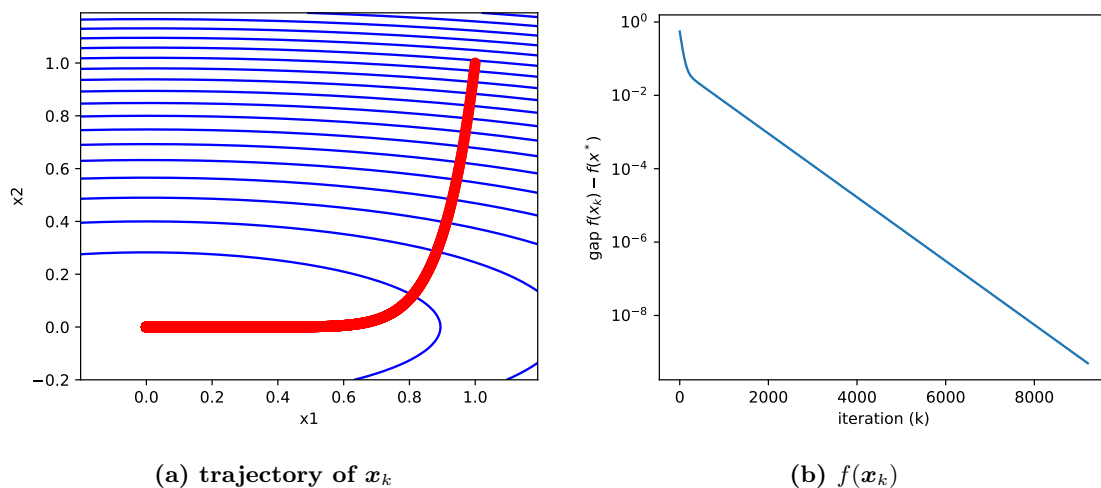


Figure 4: $\gamma = 0.1$, step size 0.01

(d). See Table 2. The number of iterations increases very fast as γ decreases. It is roughly inversely proportional to γ .

γ	number of iterations
1	1
0.1	88
0.01	688
0.001	4603

Table 2: step size = 1

□

2. Linear regression. Consider the least squares problem in Problem 4(a) of Homework 5, i.e.

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$$

where

$$\mathbf{X} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 3 \\ 2 \\ 2 \end{pmatrix}.$$

Use your implementation of gradient descent to solve this least squares problem. You may reuse the codes for Problem 1 by rewriting `f` and `fp` (you don't have to plot the traces as in Problem 1). Use any stepsize you find appropriate. Compare your solution with the solution you found in HW5, and the solution you find by solving the normal equation using `np.linalg.solve`. Do they agree with each other?

Solution. The gradient of the objective function $f(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2$ is

$$\nabla f(\mathbf{w}) = 2\mathbf{X}^T \mathbf{X} \mathbf{w} - 2\mathbf{X}^T \mathbf{y}$$

Since

$$\mathbf{X}^T \mathbf{X} = \text{diag}\{4, 1\}$$

f is 8-smooth.

One possible step size is $0.1 < 1/8$. Any step size in $(0, 1/4)$ will work (we only proved convergence for $t \in (0, 1/L]$, but it turns out any $t \in (0, 2/L)$ works).

The output is given below.

```
number of iterations: 58
solution: [1.5          1.99999521]
value: 4.0000000000229345
closed-form solution: [1.5  2. ]
```

The solution found in HW5 is `[1.5 2. ]`. All three solutions are practically the same. $\square$

3. Logistic regression. In this problem, you will train a logistic regression classifier on a toy dataset given in `p3.py`. The features are two dimensional, so the i -th sample is $(\mathbf{x}_i, y_i) \in \mathbb{R}^2 \times \{+1, -1\}$. As in Problem 2 of Homework 1, we will append a constant 1 to each $\mathbf{x}_i$ to make it three dimensional, and absorb the bias term b into $\mathbf{w}$ by defining a new $\mathbf{w} = (w_1, w_2, b)^T \in \mathbb{R}^3$. The objective function to be minimized is

$$f(\mathbf{w}) = \sum_{i=1}^m \log(1 + e^{-y_i \mathbf{x}_i^T \mathbf{w}})$$

where m is the number of samples. Its gradient is found in Problem 2(c) of HW1,

$$\nabla f(\mathbf{w}) = - \sum_{i=1}^m [1 - \sigma(y_i \mathbf{x}_i^T \mathbf{w})] y_i \mathbf{x}_i$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid function.

Find the optimal $\mathbf{w}^*$ using your implementation of gradient descent in `gd.py`. Report the classification accuracy of the classifier on the training dataset given by

$$\text{accuracy} = \frac{1}{m} \sum_{i=1}^m \mathbb{1}\{y_i = \hat{y}_i\}$$

where $\hat{y}_i = \text{sgn}(\langle \mathbf{w}^*, \mathbf{x}_i \rangle)$. You can follow the guidance of `p3.py` or write your own code. Use matrix operations to calculate the function values and gradients. You may need some of the following functions, `numpy.matmul`, `numpy.multiply`, `numpy.exp`, `numpy.reshape`.

Proof. A possible implementation of `sigmoid`, `f` and `fp` is given below.

```
def sigmoid(z):
    return 1.0 / (1 + np.exp(-z))
```

```
def f(w):
    return -np.sum(np.log(sigmoid(Xy@w)))

def fp(w):
    return -(1-sigmoid(Xy@w)) @ Xy
```

The optimal solution is $\mathbf{w}^* = (-1.47005731, 4.44147856, -4.3716255)$. The classification accuracy is 0.8666666666666667. The blue line in Figure 5(a) shows the decision boundary.

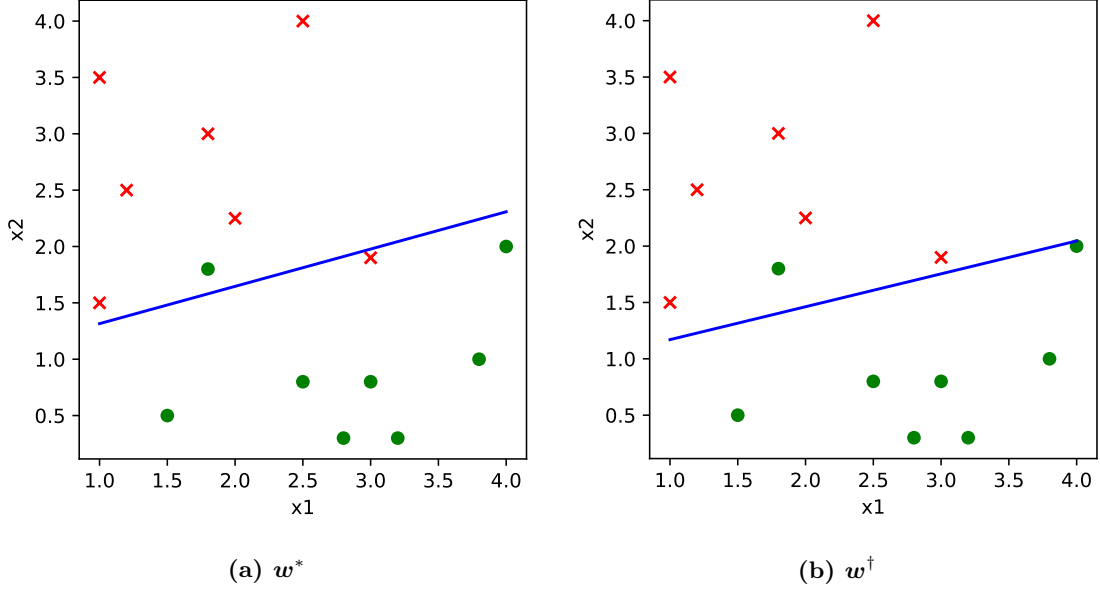


Figure 5: Logistic regression classifier.

□

Remark. The objective function f is different from the training accuracy defined by

$$\text{accuracy}(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \mathbb{1}\{y_i = \text{sgn}(\mathbf{x}_i^T \mathbf{w})\}$$

The minimum $\mathbf{w}^*$ of f does not necessarily maximize the training accuracy. In fact, for our toy dataset, $\mathbf{w}^\dagger = (-2.58627681, 8.86255152, -7.78468335)$ achieves a higher training accuracy of 0.9333333333333333, but it is not a minimum of f . The decision boundary corresponding to $\mathbf{w}^\dagger$ is shown in Figure 5(b).

4. Suppose $f(\mathbf{x})$ is differentiable and α -strongly convex, and $g(\mathbf{x})$ is β -smooth. Show that the function $h(\mathbf{x}) = f(\mathbf{x}) - g(\mathbf{x})$ is convex if $\alpha \geq \beta$.

Proof. Since f is α -strongly convex and differentiable,

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^T (\mathbf{y} - \mathbf{x}) + \frac{\alpha}{2} \|\mathbf{x} - \mathbf{y}\|^2$$

Since g is β -smooth,

$$g(\mathbf{y}) \leq g(\mathbf{x}) + \nabla g(\mathbf{x})^T(\mathbf{y} - \mathbf{x}) + \frac{\beta}{2} \|\mathbf{x} - \mathbf{y}\|^2$$

Taking the difference of the above two inequalities, we obtain

$$h(\mathbf{y}) \geq h(\mathbf{x}) + \nabla h(\mathbf{x})^T(\mathbf{y} - \mathbf{x}) + \frac{\alpha - \beta}{2} \|\mathbf{x} - \mathbf{y}\|^2$$

If $\alpha \geq \beta$, this implies

$$h(\mathbf{y}) \geq h(\mathbf{x}) + \nabla h(\mathbf{x})^T(\mathbf{y} - \mathbf{x})$$

so h is convex. □